

Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 3: Programming with Data/HTML Web Scraping

Aurélien Sallin (lecturer) and Matthias Roesti (class tutor)

27/10/2023

Exercises

Exercise A: Write a Sum Function

In the lecture on Programming with Data, we have discussed the most basic programming concepts in R. Among other things, you have learned how we can use for-loops to sum up numbers stored in a numeric vector.

1. Use the code example in the respective lecture notes and the following empty function construct to implement a function that takes a numeric vector as input and returns the sum of the vector's elements.

```
my_sum <- function(x){  
  
}
```

2. Define the vector `numbers = c(1, 2, 3)`. Then, compare your function `my_sum()` with the `sum()` function (the pre-implemented sum function in R) when using `numbers` as the input argument: Do you get the same output for `my_sum(numbers)` and `sum(numbers)`?

Exercise B: Robustness and Warnings

1. Once you have implemented and successfully tested `my_sum()`, call your function with the following vector as input. What happens?

```
numbers2 <- c("1", "2", "3")
```

2. Why do we get an error? Investigate by comparing the vector `numbers` from Exercise A with the new vector `numbers2`. Hint: the functions `str()` and `class()` might help you.
3. Extend your `my_sum()` function with a control statement that checks whether the input is of the right class for the function to work (Hint: `class(x)!="numeric"`). The goal is to make the function more robust to wrong input. That is, if the input is wrong, issue a warning but don't break down with an error. Use the following code for the warning message:

```
warning("Wrong input! This function only accepts numeric or integer values!")
```

4. Test your enhanced sum function with the two input vectors `numbers` and `numbers2`.

Exercise C: Web Scraping Exercise (advanced): Importing Data from a HTML Table (on a website)

Previously, we discussed the *Hypertext Markup Language (HTML)* as code to define the structure/content of a website and HTML-documents as semi-structured data sources. The following exercise contains the basic steps necessary to import data from an HTML table into R.

The aim of the exercise is to generate a CSV file containing data on ‘divided government’ in US politics. We use the following Wikipedia page as a data source: https://en.wikipedia.org/wiki/Divided_government_in_the_United_States. The page contains a table indicating the president’s party, and the majority party in the US House and the US Senate per Congress (2-year periods). The first few rows of the cleaned data are supposed to look like this:

```
##   year  president senate house
## 1: 1861    Lincoln      R      R
## 2: 1862    Lincoln      R      R
## 3: 1863    Lincoln      R      R
## 4: 1864    Lincoln      R      R
## 5: 1865 A. Johnson      R      R
## 6: 1866 A. Johnson      R      R
```

In a first step, we initiate fix variables for paths and load additional R packages needed to handle data stored in HTML documents. NOTE: the `OUTPUT_PATH` specified here assumed that there is a folder called `data` in the current working directory (get your current working directory by running `getwd()`) - adjust it if you that is not the case or if you want to store the output somewhere else.

```
# SETUP -----

# load packages
library(rvest)
library(tidyverse)

# fix vars
SOURCE_PATH <- "https://en.wikipedia.org/wiki/Divided_government_in_the_United_States"
OUTPUT_PATH <- "data/divided_gov.csv"
```

Now we write the part of the script that fetches the data from the Web. This part consists of three steps.

1. Fetch the entire website (HTML document) from Wikipedia with (`read_html()`).
2. Extract the part of the website containing the table with the data we want (hint: use `html_table()` on the HTML document you just fetched, and see if one of the resulting tables in the list generated by this function matches what we are looking for, you also want the option `fill=TRUE` inside that function).
3. Finally, check that the table has been properly parsed (you may want to remove the last line if it contains code junk) and store its content in a data frame called `tab`.
4. Clean the data to get a data set more suitable for data analysis. Note that the original table contains information per congress (2-year periods). However, as the sample above shows, we aim for a panel at the year level, i.e. there should be a separate observation for every individual year. The following code iterates through the rows of the data frame and generates for each row per congress several two rows (one for each year in the congress).¹ Try to understand what each line of the following code does (it should run as is if the data frame `tab` had the right format):

```
all_years <- list()
n <- length(tab$Year)
length(all_years) <- n

for (i in 1:n){

  row <- tab[i,]
  y <- row$Year
```

¹See `?strsplit`, `?unlist`, and this introduction to regular expressions for the background of how this is done in the code example here.

```

begin <- as.numeric(unlist(strsplit(x = y, split = "[\\-\\\\-]", perl = TRUE))[1])
end <- as.numeric(unlist(strsplit(x = y, split = "[\\-\\\\-]"))[2])

tabnew <- data.frame(year=begin:(end-1), president=row$President, senate=row$Senate,
                     house=row$House)

all_years[[i]] <- tabnew
}

allyears <- bind_rows(all_years)

```

5. In a last step, inspect the collected data and write it to a CSV file, using the `OUTPUT_PATH` defined at the beginning of this exercise to specify where the file should be stored.

Additional Exercises for Training (not exam-relevant)

Exercise D: Standard Deviation Function

1. During the lecture on Programming with Data we have step-by-step implemented our own R function to compute the mean. Now implement an R function called `my_sd` that computes the standard deviation, defined as: $SD = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$. Hint: revisit the math operators section in the notes on the first exercise session.
2. Test your implementation by comparing its output with the already implemented `sd()` function.

Exercise E: Standard Error Function

Building on the function written in Exercise D, implement a function to compute (the estimate of) the sample mean standard error as defined here: $SE_{\bar{x}} = \frac{SD}{\sqrt{n}}$. Hint: the idea is to use your own implementation (in Exercise D) to compute SD as a first step of your standard error function.

Exercises D and E should illustrate how you can approach more complex problems by breaking them up in smaller problems and addressing each of these step-by-step. That is, when looking at the problem of computing $SE_{\bar{x}} = \frac{SD}{\sqrt{n}}$, you see that computing SD is a part of the overall problem, which can be addressed separately.

Exercise F: T-test

Implement a function for the one-sample t-test: $t = \frac{\bar{x} - \mu_0}{SE}$.